

Deep learning price momentum in US equities

Stephen Choi
LORA Technologies
Hong Kong SAR
stepchoi@loratechai.com

Tyler Renelle
LORA Technologies
Hong Kong SAR
tyler@loratechai.com

Abstract— Research into deep learning techniques for stock price trend identification is limited. This can partly be attributed to the aversion of technical analysis within the academic community. One popular investment strategy that has been accepted by both academics and professionals, based purely on historical prices, is price momentum. However, the recent performance of this strategy has been disappointing. In this paper, we construct a new framework integrating state-of-the-art deep learning and machine learning methods to identify price trends of US equities: a “deep learning price momentum” portfolio. We first replicate the conventional price momentum calculations and compare the results with the market benchmarks and standard implementations of deep learning. We examine the issues of applying standard deep learning techniques to a *limited noisy* data set. Then we propose a new modular approach, built on deep learning clustering methods and recurrent neural networks that shows significant improvement on conventional price momentum while addressing the deficiencies of conventional deep learning methods. While the best-performing conventional price momentum portfolio yields 12.88% annual return and -0.49% market neutral annual returns for the 15-year period (2003 - 2017), our model improves these to 15.44% and +1.93% respectively with a significantly enhanced Sharpe ratio.

Keywords— *stock price prediction, stock price trend identification, clustering.*

I. INTRODUCTION

The Cambrian explosion in deep learning image recognition models following AlexNet [1] has led to convolutional neural networks (CNNs) being deployed in various industrial and professional domains. With the trading floors screens of major financial firms being dominated by price charts, it would seem inevitable for CNN models to be utilized for pattern recognition of financial asset prices. While there have been recent developments in the application of deep learning to trend prediction [2,3], extracting signals from “alternative” data such as news text [4] has garnered more interest in the machine learning community¹. Research on deep learning models purely based on historical prices is limited for a few reasons. First, unlike standard image recognition problems, asset prices are highly noisy and stochastic. Conventional image recognition is mostly deterministic, as even a child can easily distinguish between images of a cat and a dog. The identification of a stock price trend is not so clearly

defined even to the most seasoned investment professionals. Second is the inherent size limitations of the data history for medium-to-low frequency investments. *Monthly* returns across 1000 stocks for 22 years produce fewer than 265,000 monthly data points, which is significant but not necessarily *big* data, creating potential hurdles for deep learning implementation. Finally there is a general reluctance in the finance community to legitimize technical analysis - the field of forecasting prices through evaluation of past price data.

However, this does not preclude the industry from employing forecasting models based on prior prices. Within factor investing, a systematic investment approach that is well established in both the professional and academic community [5], resides the price momentum factor. Price momentum (not to be confused with the gradient descent variety) was documented in the seminal work [6], and more recently in [7] as “everywhere”, spanning global equity markets and asset classes. It is rooted in a tendency of higher performing securities over a defined period to have higher future returns during a set horizon – a simple, *linear trend*. However, as will be shown in Section III, the recent performance of the momentum factor in US equities has been quite poor.

Can we apply deep learning techniques to limited and noisy price data and identify an enhanced “deep learning price momentum” factor? Can we create this investment model without incorporating complex, unstructured “alternative” data? Our goal is not only to achieve higher returns and Sharpe ratio than the conventional price momentum and the market benchmarks but also to robustly apply deep learning to identify stock price trends with a semi-supervised data-driven approach. Proper implementation of deep learning techniques allows for flexible and efficient utilization of limited resources (such as domain expertise) compared to previous quantitative methods. A robust technical analysis study [8] requires a dedicated section defining each of the five technical patterns in the analysis specifying window size, thresholds, local maxima and minima. Studies that combine deep learning with wavelet transforms [9] require wavelet specifications. Our approach targets semi-supervised and unsupervised deep learning to the extent that is possible, with minimal externally labeled data.

In this paper, we adopt the factor framework, the standard convention for quantitative investment research, and implement recent innovations in deep learning to identify an enhanced, deep learning price momentum (tercile) portfolio of US equities. We begin with the data descriptions and test methodology in Section II. Conventional price momentum calculations are replicated, and these results are compared with

¹ Kaggle Two Sigma Using News to Predict Stock Movements Competition:
<https://www.kaggle.com/c/two-sigma-financial-news>

the market benchmarks as well as standard implementations of deep learning models in Section III. Our proposed model, deep learning price momentum (DLPM) model is introduced in Section IV, with a description of the model core components and implementation details. Section V examines the portfolio returns of the DLPM model and we conclude in Section VI.

II. DATA AND METHODOLOGY

The Russell 1000 index, representing the largest 1000 stocks and covering approximately 90% of the total market capitalization of the US stock market, is the investment universe for our analysis. The data, licensed from FTSE Russell, are composed of the Russell 1000 constituents and their daily returns from 1996 to 2017. We exclude any stocks with less than 475 daily return data in the past 500 trading days (approximately 2 years) and stocks with a price below \$1.00, as standard in financial literature [7]. This results in approximately 850–950 eligible US-listed stocks per year during the 22-year period from 1996 to 2017. The model inputs match the conventional price momentum of one-year historical prices (returns). For all implementations, the portfolio construction and data preprocessing also remain the same: at each monthly forecasting origin, we create three portfolios based on the model signals and record the one-month returns of these portfolios for the entire data period.

For every forecast origin, for each eligible stock, the model input (“X”) is a one-year history of *total* returns (to include both capital appreciation and income, e.g., dividends), and the desired output (“Y”) is the tercile classification of the following month’s (total) returns. This adheres to the factor investing framework of constructing quantile portfolios for the subsequent month ranked on model signals. The test period is a 15-year subset of the total data, from January 2003 to December 2017. The model is trained across all the eligible stocks on the historical data relative to each monthly forecast origin (e.g., the first training period is December 1995 to December 2002 and the first test period is January 2003). Then the test period is appended to the training set (e.g., the second training period is December 1995 – January 2003) and then the model is retrained on the augmented training set. This process is iterated throughout the entire 15-year period across all the eligible stocks in the investment universe. The entire test period consists of 180 (15×12) rolling monthly forecast origins.

The deep learning models incorporate one additional degree of freedom from the factor framework convention. To maintain the unsupervised ambitions of our research, we do not impose a strict one-third investment universe, equal-weight restriction on the deep learning tercile portfolios. The equal-weight, equal membership-size quantile portfolio constraints (equal number of stocks in each portfolio), require a model signal specification (e.g., probability in top quantile or probability of up or down) and portfolio construction based on the specified signal. Subsequent parameter tuning can lead to positive results but is also highly susceptible to data snooping. The DLPM model hyperparameters are either based on statistical theory or tuned during cross-validation of the training set. Furthermore, the dynamic signal-weighting schemes in [7] for conventional price momentum models yield

lower returns than the equal-weight counterparts for our investment universe during the 15-year history.

III. CONVENTIONAL PRICE MOMENTUM AND BASELINE COMPARISONS

A. Conventional Price Momentum and Market Benchmarks

The most widely adopted price (or return) momentum calculation is the 12-month cumulative return minus the most recent month’s return, a simple linear calculation, denoted as *MOM(2-12)* [7]. To construct the momentum portfolios, at each monthly forecasting origin (or “test” period) the (stock) investment universe is ranked on the *MOM(2-12)* heuristic (“model signal”) and partitioned into three equal sized tercile portfolios. We also include the *MOM(7-12)* calculations that are favored in [9]. Other linear variations of conventional price momentum fail to significantly exceed the *MOM(2-12)* or *MOM(7-12)* portfolio results.

We replicate *MOM(2-12)* and *MOM(7-12)* price momentum calculations for the 15-year period from 2003 to 2017 on our investment universe. The annualized results are given in Table I. We also include results for the Russell 1000 EW, our designated market benchmark. The equal-weighted Russell 1000 EW fits the equal-weighting scheme of the factor investing framework and yields much higher returns than the market-cap weighted Russell 1000 index. Momentum portfolios *P1* and *P3* represent the equal-weighted, top and bottom tercile portfolios, with *P3* composed of the highest (strongest) momentum signal stocks. The $P3 < P1$ relationships in both *MOM(2-12)* and *MOM(7-12)* directly contradict the factor investment objectives but follow the poor return patterns of price momentum in other academic databases². Both *P3* portfolios also yield lower annual returns than the Russell 1000 EW benchmark. $P3-P1$, which are negative for both models, are the long-short, market neutral factor returns. The $P3-P1$ factor returns aim to isolate the model factor performance from other market factors (e.g., value factor or concentration of the technology sector) that can influence the *P3* portfolio returns. It is an important measure of robustness for the model signal.

TABLE I. PRICE MOMENTUM AND BENCHMARK ANNUALIZED RETURN STATISTICS (2003 – 2017)

Model/Index	Return (Annualized)	StDev (Annualized)	Sharpe Ratio
<i>MOM(2-12) P3</i>	12.26%	15.54%	0.79
<i>MOM(2-12) P1</i>	14.01%	22.91%	0.61
Factor ($P3-P1$)	-1.75%		
<i>MOM(7-12) P3</i>	12.88%	16.77%	0.77
<i>MOM(7-12) P1</i>	13.37%	20.27%	0.66
Factor ($P3-P1$)	-0.49%		
Russell 1000	10.66%	13.45%	0.79
Russell 1000 EW	13.25%	16.89%	0.78

² Kenneth R. French Data Library: mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html

The celebrated price momentum investment strategy, with considerable assets in similar strategy products³, yields negative factor returns during the 15-year period for US equities. The table also contains the annualized monthly standard deviation and the Sharpe ratio [11] metrics (returns divided by the standard deviation of returns), a standard risk/reward measure of each portfolio performance and critical investment criterion for market practitioners. While both *P3* portfolios exhibit better Sharpe ratios than the *P1* portfolios, they only match the Sharpe ratio of the Russell 1000 EW albeit with lower returns. Can deep learning models improve on these linear investment strategies?

B. Standard Deep Learning Implementations

CNN, the deep learning standard for image and pattern recognition, and recurrent neural networks (RNN), the benchmark for sequence learning are applied to price momentum. Unfortunately, standard implementations of both models fail to provide a practical alternative to conventional price momentum. An AlexNet-based model, with more recent innovations such as Adam optimization [12] and He initializations [13], is selected as our CNN implementation. The RNN implementation employs the gated recurrent unit (GRU) [14] due to the modest structure and comparable performance relative to the accepted long short-term memory (LSTM) model [15]. The output vectors for each model are passed to a feed forward network (FFN) with a softmax classifier final layer to determine the *P3*, *P2*, or *P1* tercile portfolios membership. Multiple trials (ensembling) are necessary when performing non-convex optimization on highly noisy data and we perform 100 separate iterations of each model to ensure convergence. The statistics of the results are given in Table II. The *P3* returns for both implementations are significantly below the Russell 1000 EW and even underperform the conventional price momentum results. The CNN *P3-P1* factor returns are centered at zero (signaling randomness), and the RNN *P3-P1* factor returns are negative.

TABLE II. CNN AND RNN MODEL ANNUALIZED RETURNS (2003-2017)

Model/Statistics	<i>P3</i>	<i>P3-P1</i>
CNN model	11.93%	0.00%
RNN model	12.14%	-0.54%

The results highlight problems of CNN models in representing time dependencies. CNN models are limited in maintaining temporal information, by capturing time series as one consolidated input with positional information loss during pooling. Standard RNN implementation fares no better. The RNN models have problems of vanishing and exploding gradients [16], and even the LSTM model can face degradation with sequences longer than 50 words [17]. The granularity of our data produce long sequences: the RNN model is structured as 260 daily returns inputs (one year historical) producing an

output sequence of an additional 20 daily returns (one-month forward). Both models also face difficulties filtering for signals on a limited data set: the *entire* 22-year historical data only produce approximately 215,000 (1-year sequence) eligible samples, which might not be sufficient given the noisy data. Contrasting studies [18, 19, 20] suggest that stock market returns are neither white noise (random walk hypothesis) nor fully stationary. The financial literature in aggregate points toward a low signal-to-noise ratio in stock market returns with dynamic periods of stationarity. Thus denoising the data through conventional deep learning methods will most likely be ineffective. An optimal solution might require a more intricate, modular framework.

IV. THE DLPM MODEL

DLPM adopts a modular approach consisting of three core components: autoencoder, deep embedded clustering and RNN. The data processing begins by partitioning the entire daily return data of each stock in the investment universe into calendar month subsequences. An illustration of calendar month subsequences is included in the overview of the DLPM model process, given in Figure 1. The autoencoder then embeds the month subsequences into a latent representation of lower dimensions. Embedded clustering assigns the encoded subsequences into separate clusters through a similarity measure to transform the input data into a sparse feature space. The combination of autoencoding and deep embedded clustering, called latent clustering, is an *unsupervised* clustering process that simultaneously solves for feature learning and cluster assignment. This integrated process organizes the monthly partitioned data into similar “patterns” in an embedded feature space, creating a much more sparse representation. Latent clustering aims to preserve a significant portion of the signal in the data without maintaining long and complex sequences. Finally, each monthly subsequence is mapped to a cluster assignment and sequences of these 12 cluster assignments are passed to the RNN model. Each module is motivated by the deficiencies of standard deep learning implementations (dimensionality reduction, limited and noisy data, and temporal sequential learning) while applying well-established deep learning techniques (deep embedded clustering, autoencoding, and RNN) to meet these objectives. The modular structure also allows for management of each constraint and optimization of each task. Our Keras/TensorFlow-based implementation is available at: <https://github.com/stepchoi/DL-Price-Mom>.

A. Autoencoder

Each month is standardized into 22 “days”; we average the first two daily returns for months with more than 22 trading days (there are 23 maximum business days per month) and zero pad months with fewer trading days. The encoder maps the dimensions of the month subsequences into a latent feature space, reducing the dimensions to enhance clustering performance. Deep autoencoders [21] are used to map the 22 daily returns into a latent space with an approximately 1/3 dimensional reduction (DR). DR in financial time series is difficult, due to the aforementioned nonstationarity and prevalent noise, and ambitious DR will lead to excessive

³ Over \$15 billion is allocated to high momentum investing exchange traded funds (ETFs) according to ETFdb.com as of December 31, 2017.

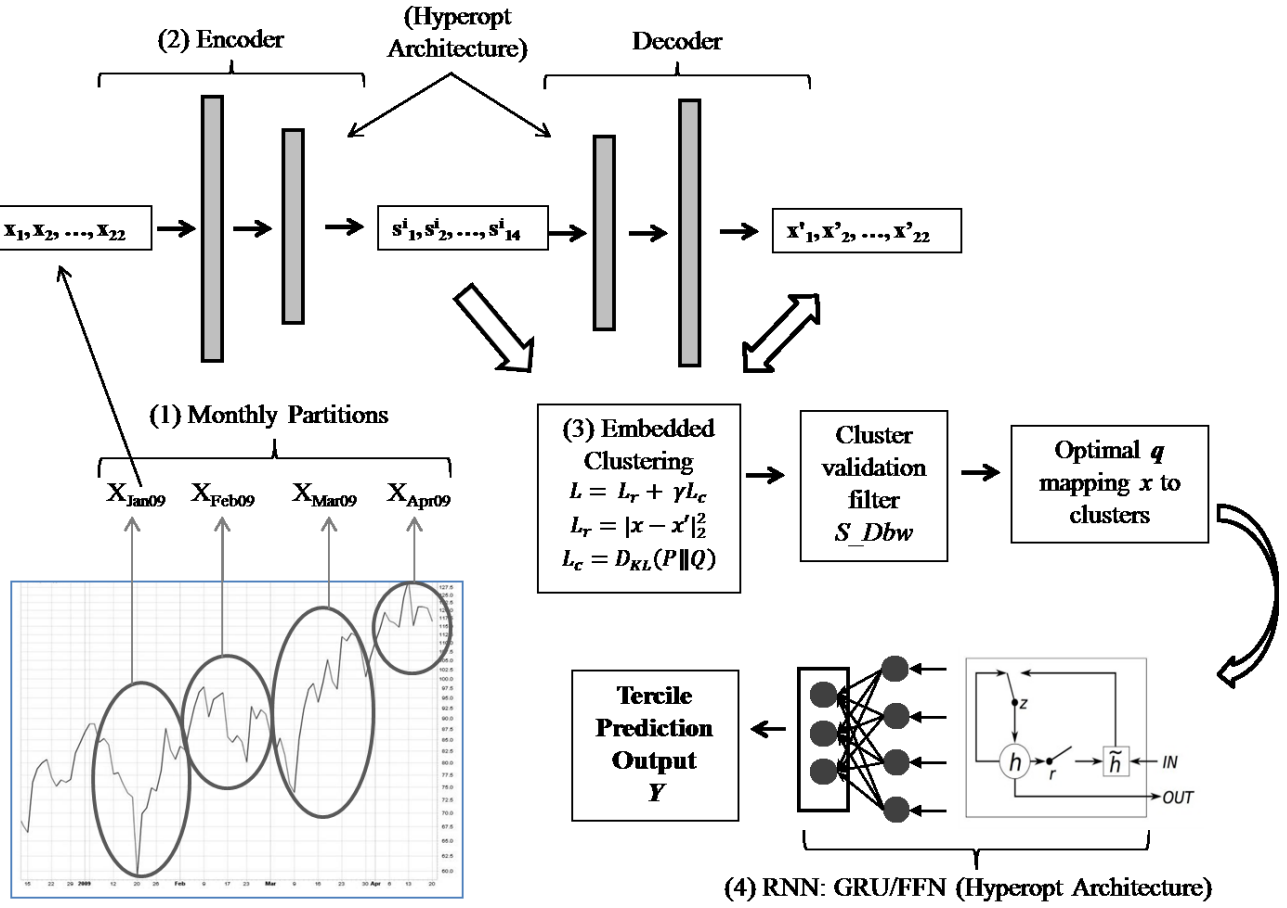


Fig. 1. DLPM model structure. The process initiates with monthly partitions of the data (1) on the lower left corner. Each month subsequence is passed to the encoder (2). Then all the encoded subsequences and the corresponding encoders go through the embedded clustering process (3) with the S_Dbw optimal cluster assignments passed to RNN (GRU/FFN) (4). The encoder and GRU/FFN architectures are selected by Hyperopt.

information loss and underfitting. We also want to avoid predetermined autoencoder architectures, to preserve the “unsupervised” spirit of our framework. Thus, we search for optimal encoder-decoder architectures through Hyperopt [22], a sequential model-based hyperparameter optimization (SMBO) interface based on Tree-of-Parzen-Estimators (TPE). At each forecast origin, we perform 50 runs through Hyperopt on a hyperparameter range of various numbers of layers, nodes per layer, learning rates and batch sizes on the monthly partitioned historical data (on training/validation split of 80/20). The tanh activation function, with consistently better results than ReLU or leaky ReLU, is deployed (this reiterates the difficulties of DR in our data and correspondingly dropout is also excluded). We select the maximum DR with an encoder architecture that consistently produces MSE (mean squared error) below a threshold (in our implementation, threshold = 0.0002). This produces 14 or 15 dimension latent representations for each 22 period (“month”) subsequences of the historical data that are necessary for optimal performance of the clustering algorithm. The encoder is trained on a maximum of 500 epochs with a reduction in learning rate and early stopping when the MSE does not improve. More recent autoencoder variations, such as the convolutional autoencoder

(CAE) are not required due to the limited dimensions (22×1) of the subsequences.

B. Embedded Clustering

The clustering scheme of the latent representations of monthly subsequences follows the deep embedded clustering (DEC) framework [22], which itself is motivated by the t-SNE models [24, 25]. We choose this unsupervised clustering framework for its success in other data sets and its ability to scale to larger data sets. In the DEC (and similar “deep clustering” methods), the Student t distribution is used as the kernel to measure similarity for soft label q , or the probability assignment of the embedded point z_i to the centroid μ_j ,

$$q_{ij} = \frac{(1 + \|z_i - \mu_j\|^2 / \alpha)^{\frac{-(\alpha+1)}{2}}}{\sum_{j'} (1 + \|z_i - \mu_{j'}\|^2 / \alpha)^{\frac{-(\alpha+1)}{2}}}, \quad (1)$$

where α is set to 1 as is standard in the literature. Unlike conventional t-SNE implementations, there is no labeled data to provide the “original” p . Instead an auxiliary target

distribution p is formed by raising q to the second power and then normalizing by the frequency per cluster,

$$p_{ij} = \frac{q_{ij}^2 / f_j}{\sum_{j'} q_{ij}^2 / f_{j'}} \quad (2)$$

where $f_j = \sum_i q_{ij}$. The soft cluster assignments are refined as q targets p by using the Kullback-Leibler (KL) divergence as the objective loss function, i.e., the clustering loss.

$$L_c = D_{KL}(P||Q) = \sum_i \sum_j p_{ij} \log(p_{ij}/q_{ij}). \quad (3)$$

By targeting P , the soft assignment distribution Q is driven to put more emphasis on high confidence cluster assignments and normalize loss contributions to prevent large cluster distortions. However, pretraining the encoder for MSE reconstruction loss only prior to the clustering tasks can lead to subpar latent representations for *clustering*; the two-stage process of feature learning *then* clustering in DEC may not be an optimal *static* combination. This is the case with our data – standard DEC has difficulties with convergence for certain samples of the encoded month subsequences. Further enhancements in embedded clustering that jointly trains the reconstruction loss with the clustering loss *dynamically* [26], such as the deep clustering with convolutional autoencoders (DCEC) model [27], are more suited for our data. A joint loss function of both the clustering loss and reconstruction loss fully utilizes the embedded clustering structure, with the clustering component driving the autoencoder to also focus on the embedded features more suitable for clustering. We adopt the objective loss function of the DCEC:

$$L = L_r + \gamma L_c \quad (4)$$

where L_r is the autoencoder MSE reconstruction loss and L_c is the clustering loss. We also follow [27] and set $\gamma = 0.1$. The gradients of the clustering loss with respect to each embedded point z_i to the centroid μ_j are identical to DEC/DCEC and joint loss minimization is performed using stochastic gradient descent (SGD) with Adam optimization. The initial cluster centers are assigned through the best selection among 20 runs of standard k-means clustering. The clustering process terminates when either less than tol of the embedded points change clustering assignments between specified consecutive t iterations (in our implementation, $tol = .001$ and $t = 5$) or at a maximum number of iterations. The maximum iterations are set as a function of the number of clusters k and data size n (in our implementation, $max_iter = 3 \times k \times \max(1e5, n)$). The early stopping on tol is examined only after 10% of the maximum iterations have passed.

The embedded clustering optimization is nonconvex and repeated iterations of clustering might not converge to the same q . Additionally, there are questions of choosing the optimal k and cluster assignment evaluation. Our solution is to perform multiple runs of clustering on multiple values of k and filter for optimal results through an established internal cluster

validation metric - the S_Dbw index [28]. Cluster assignments in q are hardened (by assigning 100% probability to the maximum for each data sample) and then evaluated with the S_Dbw index, a measure of the cluster compactness and separation. Cluster separation is the average scattering of the clusters, a ratio between the variance of the entire data set and the variance of each cluster. Cluster compactness is measured by density, which measures the number of data points that lie within a hypersphere around the cluster centroids and a radius defined as the average standard deviation of the clusters. The S_Dbw index is the sum of separation and density. Euclidian distances are applied as other distances metrics (e.g., Manhattan or Chebychev) did not result in significant differences in relative rankings. S_Dbw has demonstrated reliable validation properties consistently across various conditions [29]; we utilize S_Dbw to evaluate the explicit clustering assignments that are passed to the RNN module.

To summarize, at each monthly forecast origin, we perform a new embedded clustering process for each k between 8 and 20, and the clustering assignments with the lowest S_Dbw score are selected. Clustering below $k = 8$ or beyond $k = 20$ produces empty clusters and/or higher (worse) cluster validation scores. The k clusters in the optimal clustering represent one-month “patterns”, and each monthly subsequence across all the eligible stocks is assigned to one of k patterns as illustrated in Figure 2. This reduces the noise in our limited dataset and allows for effective deployment of the RNN. For each output clustering assignment, the S_Dbw measure evaluates the compactness and separation of the assigned subsequences and the cluster center “patterns”. While there is an overall increasing trend as we move forward in time with the expanded data, the optimal k values are not monotonically increasing (i.e., cases of lower optimal k on successive months). Thus we repeat the clustering process at each monthly forecast origin, creating a highly computationally expensive process and necessitating extensive parallelization. At the conclusion of this process, the hardened cluster assignments are then passed to the RNN.

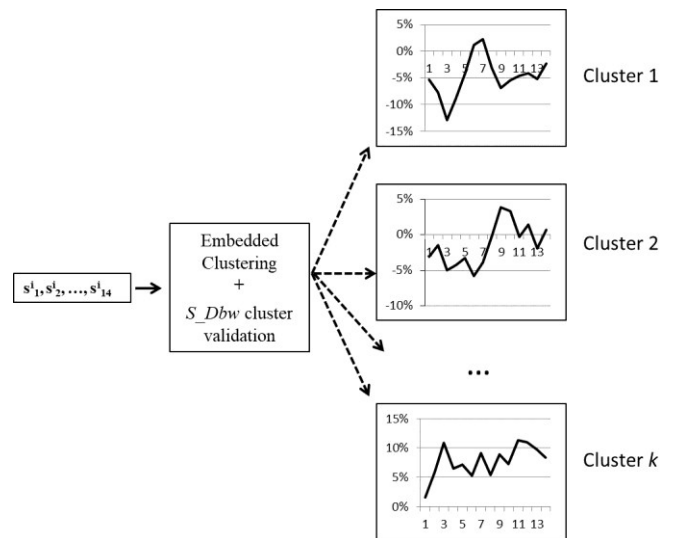


Fig. 2. The embedded clustering process assigns each subsequence to one of k “patterns”. The optimal k clustering is selected by the S_Dbw measure.

C. RNN

More precisely, 12-month sequences of the embedded cluster assignments are passed to the RNN. The standardized calendar month subsequences of all stocks in the historical data are mapped to cluster assignments from the combined (autoencoder + embedded clustering) process, i.e., each of the 22 daily return sequences representing a calendar month are converted to a one-hot encoded vector of length k . The historical data are converted to a series of 12 one-hot encoded vectors, corresponding to the one-year window of the previous models, and then are used to predict the following month's return tercile classification ("Y").

Among the RNN models, we again choose the GRU network. At each monthly forecast origin, we train on all historical 12-month sequences across the investment universe and then test on the current eligible stocks. For each stock, the output of the GRU is a vector of probabilities of length k to predict the cluster assignment of the next target month. We also augment the GRU with an FFN and a softmax classifier final layer at the top of the network to map the GRU outputs to tercile classifications. Linking an FFN directly to the GRU maintains the factor framework objective of training and forecasting the return quantile classification, not the "pattern". The Hyperopt algorithm is reintroduced to avoid preset GRU or FFN architectures. Again, at each monthly forecast origin, 100 runs of the GRU/FFN are performed on various layer depths and widths, learning rates and batch sizes for the GRU/FFN through the Hyperopt optimization with 80/20 training/validation split of the data.

V. DLPM RESULTS

The DLPM final results are given in Table III. The DLPM $P3$ portfolio annualized returns at 15.44% are higher than the top conventional price momentum $P3$ portfolio returns (12.88%) as well as the Russell 1000 EW returns (13.25%). DLPM also outperforms the returns of the conventional price momentum $MOM(2,12)$ $P1$ portfolio (the highest return of all previous portfolios at 14.01% but accompanied by low Sharpe ratios). The performance measures in Table III are annualized monthly *averages* of portfolio returns *excluding* reinvestment; the accentuated performance divergence of *compounded* investment (i.e., reinvestment of capital gains) during the entire 15-year period is highlighted in Figure 3, with the final 15-year compounded aggregate returns shown in Table IV. The DLPM $P3-P1$ market neutral factor return also improves to +1.93% and significantly outperforms those of conventional price momentum. The DLPM $P3$ Sharpe ratio is much higher than all other model results as well as the DLPM $P1$ Sharpe ratios. The robustness of the DLPM momentum factor is evident in both the returns and the performance/volatility dimension.

TABLE III. DLPM ANNUALIZED RETURN STATISTICS (2003 – 2017)

Model/Index	Return (Annualized)	StDev (Annualized)	Sharpe Ratio
DLPM $P3$	15.44%	17.35%	0.89
DLPM $P1$	13.51%	17.75%	0.76
Factor $P3-P1$	1.93%		

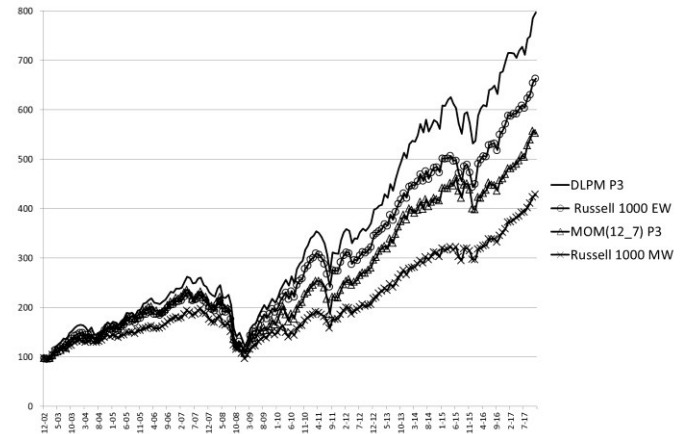


Fig. 3. Compounded investment performance (i.e., reinvestment of capital gains) of the DLPM $P3$ portfolio, the Russell 1000 equal-weight index, the conventional price momentum - MOM(12,7) portfolio, and the Russell 1000 MW index during the 15-year test period (2003 – 2007).

TABLE IV. 15-YEAR COMPOUNDED AGGREGATE RETURNS (2003 - 2007) OF DLPM AND REFERENCE PORTFOLIOS

Portfolio	15y Compounded Return
DLPM $P3$	796.64%
Russell 1000 EW	663.21%
MOM(12_7) $P3$	552.39%
Russell 1000 MW	428.30%

Given the material improvement over the conventional price momentum and market benchmarks, we can infer that the latent clustering is effective in retaining sufficient signals from the limited noisy data and that the RNN model can generate effective results from the sparse representations. However the ensemble averages of the 100 RNN output trials do not produce equal-weight, equal membership-sized portfolios. The DLPM portfolios are not conventional finance optimizations (e.g., Markowitz mean-variance optimization), but a convergence of 100 equal-weighted portfolio results from the GRU/FFN. Varying degrees of membership within the 100 outputs will result in uneven membership-sizes and weights in the $P3/P1$ portfolios. While the dynamic portfolio weighting scheme can contribute to enhanced performance, the overall membership-size weightings do not deviate greatly from strict 1/3 weighting. The average percentages of stocks in the $P1$, $P2$ and $P3$ portfolios across all forecast origins are 34.6%, 40.3% and 25.1% respectively.

If we relax the constraints further, even more gains can be achieved. Tercile portfolios are an academic factor investing convention. Market practitioners frequently employ *quintile* portfolios for investment portfolios for factor investing. To test for quintile (e.g., $P5$, $P4$, $P3$, $P2$, and $P1$ portfolios), DLPM simply adjusts the "Y" objective to the quintile classification in the training and test sets of RNN modules without repeating the encoder and embedded clustering processes. The quintile classification magnifies the model so that the $P5$ portfolio is

TABLE V. DLPM QUINTILE ANNUALIZED RETURN STATISTICS (2003 – 2017)

Model/Index	Return (Annualized)	StDev (Annualized)	Sharpe Ratio
DLPM $P5$	15.92%	17.34%	0.92
DLPM $P1$	13.22%	18.29%	0.72
Factor $P3-P1$	2.70%		

composed of stocks with even stronger DLPM signals than the tercile portfolio (and correspondingly $P1$ with even weaker DLPM signals). The DLPM quintile results, given in Table V, show 0.48% improvement to 15.92% annualized returns in the $P5$ quintile portfolio from the $P3$ tercile portfolio returns. Meanwhile, the weakest momentum $P1$ quintile portfolio returns decreased to 13.22% from the $P1$ tercile portfolio returns, resulting in $P5-P1$ factor return improvement to 2.70%. The isolated, market neutral factor has strengthened with the finer delineations of quintile classification. The $P5$ Sharpe Ratio also increased to 0.92 as well as a lower $P1$ Sharpe Ratio. The improvements in the quintile classification results across every measure only further support the effectiveness of the DLPM signal.

VI. CONCLUSIONS

A new framework for the deep learning of price momentum was introduced. The proposed architecture, accordingly termed deep learning price momentum (DLPM), is an integrated implementation of three key components. Given the difficulties of standard deep learning models on a limited noisy data presented in Section III, DLPM employs autoencoding, embedded clustering, and RNN to create a modular framework that can handle difficult stock price data. DLPM classifies and constructs optimal portfolios solely based on historical prices and significantly outperforms conventional price momentum strategies and market benchmarks. While the tercile classification of one-month forward returns is a specific factor investing objective, the DLPM model can easily be modified for alternative objectives by replacing the outputs at the end of FFN. Nonetheless, we limited the input data to a one-year history of closing prices to maintain an impartial comparison to the original price momentum benchmark. We have demonstrated favorable investment performance with deep learning techniques without incorporating a wide range of complex data inputs. As such, this is only the first phase of development and further enhancements can be made.

Additional inputs such as volume data or valuation ratios can be applied to *convolutional* autoencoding for enhanced results. Monthly partitions can be swapped with other frequencies (e.g., weekly or quarterly) and longer/shorter sequences (e.g., 6 or 24 months) can be input into the RNN module. We only focus on the S_D_{bw} index for measuring clustering performance to avoid overextending an already-complex process. Other cluster validity indices, such as the Saitta score function [30], can be evaluated for enhanced internal clustering validation. Finally, since the current composite model is computationally intensive, a form of transfer learning from one forecast origin to the next can

produce faster optimizations. These areas will be investigated in the future.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G.E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proceedings Advances in Neural Information Processing Systems 25 (NIPS 2012)*, pp.1097–1105, 2012.
- [2] C. A. S. Assis, A. C. M. Pereira, E. G. Carrano, R. Ramos and W. Dias, "Restricted Boltzmann machines for the prediction of trends in financial time series," *2018 International Joint Conference on Neural Networks (IJCNN)*, pp. 1-8.
- [3] Q. Liang, W. Rong, J. Zhang, J. Liu and Z. Xiong, "Restricted Boltzmann machine based stock market trend prediction," *2017 International Joint Conference on Neural Networks (IJCNN)*, pp. 1380-1387.
- [4] X. Ding, Y. Zhang, T. Liu, and J. Duan, "Deep learning for event-driven stock prediction," in *Proceedings of the 24th International Conference on Artificial Intelligence (IJCAI'15)*, pp. 2327-2333.
- [5] E. Dimson, P. Marsh, and M. Staunton, "Factor-based investing: The long-term evidence," *The Journal of Portfolio Management*, vol. 43, no.5, pp. 15-37, 2017.
- [6] N. Jegadeesh and S. Titman, S., "Returns to buying winners and selling losers: Implications for stock market efficiency," *The Journal of Finance*, vol. 48, no.1, pp. 65-91, Mar. 1993.
- [7] C. S. Asness, T. Moskowitz, and L. H. Pedersen, "Value and momentum everywhere," *The Journal of Finance*, vol. 68, no. 3, pp. 929-985, June. 2013.
- [8] A. W. Lo, H. Mamaysky, and J. Wang, "Foundations of technical analysis: Computational algorithms, statistical inference, and empirical implementation," *The Journal of Finance*, vol. 55, no. 4, pp. 1705–1765, Aug. 2000.
- [9] J. Jin, and J. Kim, "Forecasting natural gas prices using wavelets, time series, and artificial neural networks. *PLOS ONE* vol. 10, no.11, 5 Nov. 2015.
- [10] R. Novy-Marx, "Is momentum really momentum?" *Journal of Financial Economics*, vol. 103, pp. 429-453, 2012.
- [11] W. Sharpe, "Mutual fund performance," *The Journal of Business* vol. 39, no. 1, pp. 119-138, Jan. 1966.
- [12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations*, 2015.
- [13] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proceedings of the 2015 IEEE International Conference on Computer Vision (ICCV)*, pp. 1026-1034.
- [14] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder-decoder approaches," in *Proceedings of Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation (SSST-8)*, pp. 103-111, Oct. 2014.
- [15] S. Hochreiter, J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol 9, no. 8, pp. 1735–1780, Nov. 1997.
- [16] R. Pascanu, T. Mikolov, and Y. Bengio, "On the Difficulty of Training Recurrent Neural Networks," in *Proceedings of the 30th International Conference on International Conference on Machine Learning*, pp. 1310-1318, 2013.
- [17] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proceedings of the 27th International Conference on Neural Information Processing Systems (NIPS 2014)*, pp. 3104 -3112, 2014.
- [18] E. F. Fama, "The behavior of stock-market prices," *The Journal of Business* vol. 38, no. 1, pp. 38-105, Jan. 1965.
- [19] R. Thaler, "Anomalies: The January effect," *Journal of Economic Perspectives* vol. 1, no.1, pp. 197-201, Summer 1987.
- [20] A. W. Lo and A. C. MacKinlay, "Stock market prices do not follow random walks: Evidence from a simple specification test," *The Review of Financial Studies* vol. 1, no. 1, pp. 41-66, 1988.

- [21] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P. A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *Journal of Machine Learning Research* vol. 11, pp. 3371–3408, Dec. 2010.
- [22] J. Bergstra, B. Komer, C. Eliasmith, D. Yamins, and D. D. Cox, "Hyperopt: a Python library for model selection and hyperparameter optimization," *Computational Science & Discovery*, vol. 8, no. 1, July 2013.
- [23] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *Proceedings 33rd International Conference on Machine Learning*, pp. 478–487 June 2016.
- [24] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605 Nov. 2008.
- [25] L. van der Maaten "Learning a parametric embedding by preserving local structure," in *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*, vol 5, pp. 384–391, 2009.
- [26] B. Yang, X. Fu, N. D. Sidiropoulos, and M. Hong, "Towards k-means-friendly spaces: Simultaneous deep learning and clustering," in *34th International Conference on Machine Learning*, Aug. 2017.
- [27] X. Guo, X. Liu, E. Zhu, and J. Yin, "Deep clustering with convolutional autoencoders," in *Neural Information Processing. ICONIP 2017. Lecture Notes in Computer Science*, 10635: 373-382. Springer, Cham. 2017.
- [28] M. Halkidi and M. Vazirgiannis, "Clustering validity assessment: Finding the optimal partitioning of a data set," in *Proceedings of the 2001 IEEE International Conference on Data Mining*, IEEE Computer Society, Washington, DC, USA, pp. 187-194, 2001.
- [29] Y. Liu, Z. Li, H. Xiong, X. Gao and J. Wu, "Understanding of Internal Clustering Validation Measures," 2010 IEEE International Conference on Data Mining, Sydney, NSW, pp. 911-916, 2010.
- [30] S. Saïta, B. Raphael, and I.F.C. Smith "A bounded index for cluster validity," in *Proceedings of the Machine Learning and Data Mining in Pattern Recognition. MLDM 2007*, Lecture Notes in Computer Science, vol 4571. Springer, Berlin, Heidelberg, pp 174-187, 2007.